Introduction

Hello, everyone. And again, welcome to lecture 6 of CME 295. So today is actually an exciting day because we're going to cover a topic that has been trending over the past year or so which is LLM reasoning. And it's actually a good segue compared to what we talked last time, which was preference tuning, because a lot of the methods that we used in lecture 5 are going to be the ones that we'll be using as the foundations for this lecture.

So before we start, as usual, we're going to just cover quickly what we saw in the last lecture. So if you remember, lecture 4 and lecture 5 were all about learning how we can train a model. So in lecture 4, we saw the first part, which was the pre-training part, which is the most compute-intensive step where we basically teach the model the structure of text, the structure of codes. And we do this very large, large scale training. So at the end of this first step, which is pre-training, we get a model that knows about code, that knows about language, but it only knows how to autocomplete a sequence. And so that's why we also saw the second step, which was the fine tuning step where we take our pre-trained model and we try to make it useful. So one use case that we saw was for instance, an assistant. So we tried to tune it in a way that it can respond to questions. And so here, we have this work of preparing the SFT data, which is what we call the SFT data, which is a high quality, curated data set that we can use to teach our model on how to behave. So at the end of this second step, we have our model that is tuned for a specific task, which can be responding to queries. And then last lecture, we saw this third step, which was the preference tuning step. And here the goal is to align our model with human preferences. So we saw, in particular, RLHF, which was a common method to do that. And we saw that there was two steps to it. So there was the first part, learning to distinguish good from bad using human preference data, and then there was this second step, which was this RL stage, which is going to be useful today.

So in particular, if you remember, we had drawn a comparison between the RL setup that you may be familiar with outside of this class, where we have an agent

that is interacting with the environment and the way that it interacts with it, is that given a state that it is in, it can take an action following a policy, which is nothing else than just a probability distribution over actions. So given a state, our agent can take an action following this policy. And at the result of this, it receives some reward. And we saw last lecture that we have this nice comparison between the traditional RL setup and the LLM setup. And so here, our, quote unquote, agents is just simply the LLM. The environment that it interacts with is just a set of tokens that it can predict over. So given an input that it has received so far, it can predict what the next token could be, which is the action. And it does that using the probability distribution that is the result of, I guess, the LLM prediction. And we saw that we can obtain human preferences for each completion. So you have a prompt, you have a completion, and you get those human preferences. And this is the one that you then use in order to tune the LLM. Because here, this step is about aligning the model with human preferences. And here, the human preferences is encapsulated in the reward. And so we saw that the loss function during the RL stage is composed of two parts. So the first part is this advantage maximization. And we saw that that advantage was based on the rewards. And it has some baseline to just reduce the variance of the gradient. So we have that part. But we also saw another part, which was that we don't want our model to deviate too much from either the previous iteration-- so we don't want the model to change too much from iteration to iteration-- but we also don't want our model to change too much compared to our initial model, like our base model. And here, it's the SFT model. And the reason why we don't want to change too much is because our model has already learned a lot. It's already quite performant in what it does. And what we want is to just align it with human preferences, which is not something for which we want the model to change completely.

And I think that was the part that I think was a little bit scary, which was the actual loss functions. So if you remember, the main algorithm that is typically used in the RLHF setting is PPO, which stands for proximal policy optimization. And there is this one variant that's called PPO CLIP, which is such that it clips the updates from one iteration to another. So here, if you remember, R is actually not the reward, it is the ratio. And it's a confusing notation. So it's the ratio between the

current policy and the old policy. And the old policy here is the policy that is at the previous RL iteration. So what we do is that we have a clipping mechanism such that the ratio cannot go beyond some certain thresholds, such that we do not want to incentivize the model to make too big of an update. And we saw another variant of PPO which is called PPO-KL penalty. And this variant uses the KL divergence to penalize the model from changing too much. So if you remember in the original PPO paper, we used the old, quote unquote, old version of the model, which is the one that the previous RL iteration. But in modern day RLHF training, this KL divergence is typically applied to the base model, which is the SFT model. So I guess long story short is we have these two PPO variants that were variants that were introduced in the original PPO paper. But in modern day RLHF training, we typically have some combination, some mix of these two loss functions. Cool?

So up until now, we've seen what I want to call vanilla LLMs, which are LLMs that take something as an input, let's say a prompt, and just respond with some answer. And so those vanilla LLMs, they have a lot of strength that you can enjoy. So first of all, we had seen that those LLMs, they know a lot about structure, of the text, they know a lot about codes. So in particular, if you want to debug your code, I guess they are great to find where the error is. They are great to generate codes. They're also great to generate essays or poems. They're really, really good at that. But I do want to call out some weaknesses. So the first, I guess, weakness that I want to call out in this vanilla LLMs is that they have, quote unquote, limited reasoning. So typically, if you have some sophisticated, let's say, math problem, it will not really come up with, I guess, the solution because maybe it will get lost in the way because up until now, our model has been really trained to, I guess, given a prompt, respond to it using the next token prediction. So I guess here, there is not really a big reason why it would be able to solve complicated problems. So I guess that is one. So second weakness is that the LLM that we have has been pre-trained on a huge amount of data, which is static, meaning that the knowledge that the LLM has acquired is bound to the cutoff date, what we call the cutoff date, at which we cut and formed our pre-training data. So I know a few days ago, we had an election. So if, let's say, we trained our LLM based on data before the election and let's say today, we ask it, who is the--

I don't know-- elected official of let's say X, it will not be able to answer us because it does not have access to knowledge after that date. So a third weakness is so far, it's all talk, no action. So you just prompt your LLM. But I guess if you want to let's say, I don't know, place an order or don't do some action, you just kind of do it. And then I guess the last weakness, which by the way, is not an exhaustive list, is contrary to traditional NLP models LLMs, they generate free form texts. And it's hard to evaluate them in the framework that we-- I mean, by we is the ML community has adopted up until a few years ago. So if you're familiar with it, let's suppose if you worked in the translation world, you would use rule-based metrics like BLEU or for summarization, ROUGE to evaluate your output. But then here, LLMs, they can really do more than just that. So it's very hard to evaluate them. So what I want to say is that this is, let's say, a subset of all the weaknesses that LLMs can have. And the last three are going to be topics that we'll cover in the next lecture and in the lecture 8. And as I mentioned before, the focus of today is reasoning. So we'll see how we can improve the way our LLM reasons.

Reasoning models

Cool. And again, this is a topic that's been very new. And by new, I mean roughly a year. So almost everything that we will see is either from 2024 or 2025. And I guess we're lucky because now we have enough hindsight to just know which piece is more important than other things. And so the goal for today is going to be to know what reasoning models are. And the second big goal is to how they are trained. So hopefully, at the end of this lecture, if you have a good answer to these two questions, that means that we have done a good job.

So let's start with reasoning models. What are they? Well, to answer this question, we first need to, I guess, define what reasoning is. And the bad news here is there's not a commonly agreed upon definition out there of what reasoning is. So I will try to define it, I guess, to the best of my ability. So here, we define reasoning as the ability to solve a problem. And here, by problem, we typically think more of math problems or, let's say, coding problems. But hopefully, these abilities can also spread to other fields as well. And in order to

solve this problem, we typically need a multi-step reasoning process, a little bit like when you take an exam, when you have a question that's not trivial, you typically break that down into several steps and then go through them, and then come to the final answer. So I guess the idea is problems that are reasoning problems, they would have some pattern of that.

So to illustrate this, let's just make sure we're all kind of thinking the same. So a non-reasoning question would be something what is the course code of Stanford's transformers and LLM class. So this one, it's a knowledge thing. Everyone knows it's CME 295. But then as opposed to that, in contrast to that, a reasoning-based question would be maybe a math question. So for instance, let's suppose we have a bear that was born in 2020. How old is that bear now in 2025? So that would be the kind of question that we're looking at. Of course, this is super easy. You can think of something that's much harder than that, and this would qualify.

Cool. So now that we a little bit what reasoning is, now we're going to look at how we can, I guess, obtain a model that can deal with these kinds of prompts. So the core idea here is to leverage a concept we saw earlier in the class. So I'm not sure if you remember, but back in lecture maybe two or three, we saw a technique called chain of thought. So who remembers what chain of thought is? Yeah? Yes, do you want to say what that is? Yeah, exactly. So the answer is thinking in steps instead of just giving just a blanket answer. So great answer. So here, just to illustrate-- yeah. Oh, yeah. Great point. So the question is, for some questions, you need to have some element of context. So for instance, here, you need for your LLM to know that this year, I don't know, today is November 7, 2025. So yes great points. So there are two parts of that. For that specific question, so typically LLMs they have something that we call preamble. So something that tells you about some context. And typically the date is something that we put in the preamble. So for this very specific case, the date would be information that would already be, I guess, available to the LLM. But for some other problems, you can very well have information or context that you do not have. And in lecture 7, we will see how we can fetch that. And it's definitely something that will be very useful. But for the sake of today, we will consider this more as being an extension

of reasoning models as opposed to something that is foundational to how they work. But we'll respond to this part, I guess, next lecture.

Cool. And so here, we are about to illustrate what a chain of thought looked like. And so here, instead of having, as you mentioned, just a blanket answer, what we want is to explain also the reasoning. So the way chain of thought did that was by having some in-context learning examples that explicitly the reasoning to encourage the model to also do that before providing the answer. So that's the rough-- it's not the rough idea. It's the idea behind chain of thought. And here, what we want to do is to do that, but at a much larger scale. So I just want to build an intuition as to why this may help us. So LLMs, they are basically trained with this next token prediction objective. And so the way they respond to questions that we ask is typically to sound plausible in a way to maximize or optimize for the probability of the tokens that you want to generate to happen. So if you have a very hard problem that you're presenting to the LLM, there are very few chances that that problem appeared in the training sets. And so here, the idea is to have the LLM decompose the problem into tractable ones, and then rely on the patterns that it has seen during training to solve all these more tractable problems. And it's a little bit like you or when we were students. When we give you a problem, you try to link it back to something that you know, that you have seen at training time, like during your studying, to solve them in order to find the answer. So I guess that's the intuition. I guess another reason that you can think of is when you let the LLM generate more tokens, you're just giving it more compute, if you think about it, because at each generation step, you have this whole forward pass. And when you do that more, you just give it more compute. And we will see there is a term that is dubbed compute budgets, which is, I guess, the budget that you want your LLM to have to generate your response that we will see later on. But yeah, so that also plays a role.

So is everyone clear on the overall idea of what a reasoning model is? Yeah? Perfect. So just to make sure, again, that we're all clear on this, so up until now we had, quote unquote, Vanilla LLMs that had some question as inputs, and they were responding with something as output. And in our case, what we want is given a question as input, we don't want to directly generate an answer. We first

want to think. And here, I guess we think by first outputting a reasoning chain. And then we provide the answer. So here, the LLM output is not just the answer. It's the reasoning plus the answer.

And I guess here, I was saying that this topic has been very hot and trendy for the past year. So this is a little bit of a preview of the timeline of reasoning models. So reasoning itself is a topic that has been studied for more than a year, but reasoning models have started popping out starting from OpenAI's release of 01 preview. And this one was September of 2024. And then after that, you basically have everyone just wondering how OpenAI did it. And so you had all of these AI labs trying to see what we can do to, I guess, increase the reasoning abilities of the model. So you would have everyone working on this. And then you had Google's Gemini 2.0 flash thinking that was, I think, released in December. And then starting in 2025, there was this DeepSeek R1 paper that was published in January, which made a lot of noise because what they were able to do was to match OpenAI's reasoning ability performance with a method that they were actually describing in their paper. So that was a big moment, January 2025. And then after that, all of these other models also had reasoning abilities added. So you had some models from xAI, from Anthropic with Claudes, and then from other labs as well, also from Mistral. So this timeline is not exhaustive, but it's just to show you, I guess, how recent these models are. And I guess one thing to have in mind is it basically started at the end of 2024.

Cool. So I know a lot of you, if not all of you, are using LLMs every day as chat bots like, let's say, ChatGPT or Gemini. And so I want you to know, when the model that you're interacting with is a reasoning model-- so I have a question for you. I was having a discussion with ChatGPT the other day. And I was wondering, is this using a reasoning model. So what do you think? If you stare at this screenshot. Yeah? Why? Right, exactly. So the key word here is thinking. Actually, they put it everywhere. But when these UIs show you the thinking process, so what they do is actually-- so they don't show you actually the full reasoning process. And we're going to see why. But they tell you that they spend time thinking. And that time thinking is actually the time that is needed to produce the reasoning chain. That can be, I guess, more or less something that takes time. And so for instance,

for ChatGPT, you have this thinking that you can also change from standard to extended, et cetera. And you have this for other models as well. And in particular, the, quote unquote, thought summary that you see is not the raw reasoning chain. It's a summary of it. And the reason why they do that-- I mean, I'm hypothesizing-- is one, because the raw chain may not be something that is fully intelligible from a human standpoint. Second is because maybe you as a user, you don't want to read the pages and pages. And then third, this is going to be what we see. But if you have the reasoning chains, you can maybe train a model that is trained on these chains so it can basically mimic those abilities as well. So this may be the reasons why you would typically not see the raw chain.

And when it comes to pricing, so we saw that the output of reasoning models are not only the answer, but also the reasoning itself. And so you will see in, I guess, all of these APIs that you're actually getting charged for it. So you're not necessarily getting all the reasoning chain. But if you look at the docs, there's always, I guess, a sentence or two, I guess, that specifies that you're actually being charged for output tokens. And those output tokens, they also include reasoning tokens. So I think that's also a good thing to know. And I guess from a user standpoint, this also gives an incentive to have the maximum reasoning ability for a minimum amount of reasoning token because you don't want to pay a lot. So we're going to see later on how we can deal with that. But this is just one thing that's good to note.

Benchmarks

Cool. So we talked about what reasoning was, or at least a tentative definition. We saw, I guess, the core idea behind reasoning models. So now we're going to talk about some of the benchmarks that people use to quantify reasoning abilities. So the first one, as I previously mentioned, is about assessing the coding abilities. And so here, the goal is typically to solve a coding problem or to fix a bug. So you typically have the following setup. So given a problem, you want to produce a solution that is able to pass test cases. So if you have a solution that passes all test cases, then you have a solution that works. And this is our way of verifying that a response that was generated is actually a correct one. So

that's for coding. So just giving you an overview of the kinds of benchmarks that are out there. So we have HumanEval, which is the screenshot screenshots here, which I believe is a set of 100 something coding problems that were human written, which is why it's called the HumanEval. But you have then CodeForces, which is coming from a website of competitive programming that some of you may know. And there is also SWE-bench, which is, I believe, a set of problems that were derived from GitHub issues. So these are real practical problems. So you will see that these reasoning models in the reports, they typically have results along these benchmarks. So this is for coding.

Now for math, the goal is to solve a problem. And the way it works here is given a problem, you want the answer. And of course, you're letting your model, generate some reasoning. And so here, what you would do to verify that your solution works is by parsing the answer like so and then comparing it with some ground truth. So here, we can really know if the answer that your model is producing is true by comparing the two. So you will ask me OK, how do you parse that. So you can force your model-- and by force, I mean in the prompt-- to output the answer in a way that can be parsable. Sometimes people, they just put it in some box in the box brackets. So there's a number of different ways to do that. And this is how the problems look like. So you have some problem. And then you can let your model generate some reasoning chain along with the answer. And then you have the answer here that you can compare to.

So I just want to add that the kinds of benchmarks that are out there are based typically on problems that are actually not that trivial. So a name that you will see a lot is aim. So I'm not sure if you're familiar with it. It's a math exam to qualify for the US math olympiads. So there is a bunch of models that I guess quantify their performance based on that. There's also GSM 8–K, which is a grade school kind of math problem. And I guess now your question maybe, OK, it's great we have all these benchmarks, but what is the metric that you will use to quantify that what you're doing is good or not.

Pass@k metric

And so there is one metric that you will see a lot. And we're going to see that right now because it's not that obvious. So that metric is called pass@k. So pass@k is, by definition, a metric that tries to estimate the probability that at least one of k attempts succeeds. So here, what we're saying is let's suppose we're, I don't know, having some coding problem. We tell our model to generate k answers. And pass@k is the probability that at least one of these k answers passes the tests. Does that make sense?

So by the way, why would you have pass@k? Why would that make sense? So it's not super trivial. So you may have used cases where you can afford to spend more time to generate more answers. If you know that you will get more chances to get it right, then you can spend more time to generate more answers so that you can have the right one. So in a problem like coding, the good thing is you can check whether your answer is correct or not. And so here, the idea is if you are in cases where you can afford to generate not just one answer, but multiple answers, then it may be worth it to just spend more time, spend more compute to generate more answers if it means that the probability of getting it right in one of them is going to be higher. There is one technique that we saw last lecture that is kind of familiar with this. Best of n. Do you remember? Yeah, roughly. So best of n is this method that we saw last week, which was you generate n answers. And you use your reward model to score everything. And you take the best of all of them. So here, you can think of it as being the same, just that we do not have a reward model, we have a deterministic, verifiable way of checking whether something is correct. So it's very similar to that.

And so now we're going to spend just a few minutes just aligning our understanding on how we can estimate such a quantity. Yeah. That's a great question. So the question is, do you choose a particular temperature. So I have a slide on that. So I'll just hold your question for a few slides. Cool. By the way, if there is any other questions, happy to-- is there any other questions on this? Everyone is super clear? OK, cool. So what we want to do here is to estimate this probability. But you may tell me, just generate k attempts and just count the number of the attempts that are correct to estimate that. But the problem is if

you only do this k times, your estimate can be very noisy because by pure chance, I don't know, you may have one instance that has 3 correct out of 5 and the other one 1. So you want to have an estimate that doesn't have that much variance. So what you do is you typically generate not k, but n answers. And out of these n answers you're going to have c of them that are going to be successful, and then n minus c that are not going to be successful. And the question here that you ask yourself is out of these n attempts, you want to quantify the probability of having at least one out of k attempts that is right. So the question here, if you have those n observations, if you were to select, let's say k, what is the probability of having at least one of those k passing? That's the question. So we're short on time. I'm going to actually derive that right now. And the reason why I want to do that is because the answer is not necessarily trivial. And I don't want you to be too surprised of how it looks like without knowing where it comes from.

So we're going to derive that. So we want to derive the probability that at least one attempt out of k that we select out of these n samples is correct. So in order to do that, I'm just going to write that down. So pass that k. So we want the estimate for that. So it's going to be the probability that at least one attempts out of k is correct. So if you've done some probability, there is a very common trick, which is if you have a probability of at least one, it's 1 minus the probability of having all of them being incorrect. Do you all know that trick? Because we're going to use that trick. So it's going to be 1 minus the probability of having all k attempts incorrect. So far so good? Yeah? So now we're going to try to quantify this probability. So what is the probability of you finding an unsuccessful attempt among these n you have n minus C unsuccessful. You have n observations is going to be n minus C over n. So it's your first it's your first unsuccessful attempt. So now what is the probability that you have another unsuccessful attempts knowing this one? So you have n minus c minus 1 other unsuccessful attempts over n minus 1 because you already took one. And you continue that k times n minus c minus k plus 1 over n minus k plus 1. Does everyone agree with me? So here, we are quantifying the probability that if we take k samples randomly among this n, that all k1's are incorrect. So here, we compute this by computing the probability that the first attempt is incorrect.

And then the second one is incorrect, knowing that the first one was incorrect, and so on, which we have for which we have this formula. So if you saw that in probability class, it's basically the same as sampling without replacement. And now the nice thing with this is that you can express that with a nice mathematical notation So for people who know, so n choose k is equal to factorial n factorial k factorial n minus k. So this is by definition of what this quantity is about. So we're going to use that here. So here, we have a series of products. So here, we can express that as factorial over another factorial. So here, it's factorial n minus c over factorial n minus c minus k. And then this quantity, we can also express that with factorials. So it's factorial n. And the numerator is factorial n minus k. Everyone agrees with the math here? I'll take that as a yes. And so here, we're going to somehow pop the expression above up. So it's going to be equal to 1 minus so n minus c factorial over n minus c minus k factorial. And then k factorial. And then here, we're going to say it's this one, k factorial k factorial over n factorial. So here, I did a trick. What I did was I basically multiplied the numerator and the denominator by k factorial. And so here, I have 1 minus-- so this is n minus c over n. So n minus c choose-- sorry, n minus c choose k over n choose k. So this is our estimates of path at k. Does everyone agree? Yeah?

So the reason why I derived it is because this formula can be a little bit daunting to look at. But it's actually quite natural because it's only using some sampling without replacement considerations. And so you will see in papers that people have this pass@k metric. So if you were to compute it, this would be the formula to use. Cool. And special k of pass@k is pass@1. And pass@1 is defined as the probability of a single attempt succeeding. So this is something that you commonly know. So if suppose you have a model that produces an answer, what is the probability that it's correct? So if you replace k with 1, you will see that this formula will simplify a lot. And it will be just a proportion of successful attempts, which also makes sense, just intuitively. Everyone clear with this? Yeah? Great.

So now to your question. So what do you do with the temperature. And you're exactly right. When you generate these solutions multiple times, something that will influence your results is how diverse the solutions that you will generate will be. And this is something that you indeed use the temperature for. Now so what

is the relationship between temperature and pass@k? Do we want low temperature, do we want a high temperature, or do we want something in between? Well, if you take a very low temperature, you know that your solutions will be good, but they will not be diverse. So if you increase the number of samples that you generate, that quantity will not change, which is seen with this graph with the T equal to 0. It doesn't change. But then when you have T equal to 0.2, now it's increases because you have some diversity. But if you increase this temperature too much, then you will have the diversity that you want. But they will also harm the performance of your predictions, because maybe the tokens that were not that likely will become likely. And so here, on the opposite side, we have t equal 1.2 here, which is extreme in this example, which is not the best. So I guess to respond to you, typical temperature choice would be something that is not too large, not too small. And I believe in this example is t equal to 0.8 that I think produces the best results towards the bigger samples. But then here, I guess it's not clear. Maybe 0.4. So that's why you will see that in papers, people always specify the temperature that they chose to produce the benchmark results. So that's why it's good to have that in mind.

So these are the main metrics that people use but not the only ones. There's another one that you may see, which is called consensus at k, which is the answer that is coming from the highest number of times that the answer appears among your generations. And you can think of the self-consistency technique as being very related to that. So you may see that. And then the other metrics that you will see in those benchmarks are typically the ones that you're familiar with like accuracy, exact match, and so on.

Scaling with RL

I took a lot of time on this first part. I think I need to go a bit faster. But does that roughly make sense so far? Yeah? So right now, I hope you what a reasoning model is. So now we're going to go into the most interesting part of the lecture, which is how can we build such a reasoning model. So we know we want our reasoning model to produce a reasoning chain. But right now, we don't really how to do that at scale. And this is going to be the main focus in this part. So here

the idea is to somehow incentivize the model to produce a chain of thought or reasoning chain before answering. So the problem with that is that reasoning chains writing is a very tough task, especially for, I guess, long reasoning chains. And for that, let's suppose if we were to go look into our toolbox of the techniques we have learned so far, maybe we can think, OK, we can use SFT to do that. But the problem with SFT is you need high quality data, and in particular, you would need to write all of these reasoning chains. Let's suppose you do not have this reasoning chains. So how do you do that? You would typically write them from scratch. But that is very hard. So that's one, I guess, one vote to not do SFT, if we don't have any reasoning chains. So the second reason here is, or the second fact, is that the way the model reasons may be different from the way we reason. And for that, maybe a human written reasoning may not be the best way to teach the model on how to reason. So that's the second fact, which is going against having some human written reasoning chains as SFT data. And then the third fact that I want to say is we just saw that these reasoning tasks, they have a very natural reward, which is actually verifiable. So for coding, you know if your thing works, if it passes some test cases, if it compiles, and for math, you know if it works if the answer is the same as your ground truth. So naturally, you do have a reward signal. So we saw there's a reward signal. SFT is not a great way to start from scratch. Well, what do you do? Well, let's try RL then. And the good thing is last week, we saw how RL works in the case of LLMs. So I guess that's a great timing.

So how would we do that? So just as a reminder, we want to teach our model to solve these more complicated problems. And we also want to teach it to reason before. So in order to do that, we first want to have a reward that checks whether the reasoning chain is there. So here, you can do that by simply checking whether the model has produced a reasoning chain. And you can check that by checking the presence, let's say, of the think, start, and end token that you can instruct your model to use to insert its reasoning chain. And then the second reward that you want to have is to check that the solution that it produced actually works. And so here, as we mentioned, this is something that you can actually do for the benchmarks that we care about. So for codes, you would check that it works for all cases. For math, you would just verify with respect to

the answer being the same as the ground truth. So we also have that. So in summary, we will run RL, and we will run RL using a reward that is a combination of checking whether the thing tokens are here and that the end result is correct. And we saw that both of them, we can do this easily. No need for a model.

So let's suppose you do that. Let's suppose you do RL on search rewards. Well, the nice thing that you see is if you plot the performance of the model with respect to one of those benchmarks that I mentioned-- so here it's aim. So if you remember, it's the math benchmark-- you see that as you perform your RL steps, the performance of the model on those benchmarks, they actually increase. And they actually increase quite significantly. So the graph here is actually coming from the training of DeepSeek R1-- actually, R1–Zero here, which we will see a little bit more in detail with Shervine how it actually works. But you can see that if you incentivize the model to think and produce the right answer, it can actually learn quite meaningfully, only with these two verifiable rewards.

But now you may wonder, let's suppose I'm teaching my model to think, well, not all prompts are equal. There are some prompts that do not need the model to think too much as opposed to other prompts which would need a little bit of thinking. So there has been a lot of, I guess, work happening in the community as how to control the amount of thinking that the model does. So you have things like dynamic budget that you will see out there, how can we make sure that the model does not overthink on questions that do not need too much thinking. So for this, you can have something like a quick classifier, let's say, that runs on the prompt and tells you whether it's something that is kind of a high thinking kind of problem or a low thinking one. But this is more of an open problem. But this can be one solution. So the second one is those LLMs, they have a limited context window. So when the LLM thinks, it needs to. Also be aware of how much context length is left. It cannot think too much also because of that limitation. So you need to also have that context awareness. So there has been some work to also force the model to either think more or stop. So there is this term, budget forcing, that I think was introduced in this paper, S1. So the idea here is if you want your model to continue thinking, you're going to introduce some tokens mid-way to force it to think more. And such tokens are, for instance, weights. I

think there is another solution. So actually, it's a slide that I have not really talked about. So if you look at this reasoning chain, sometimes the model will output something like, oh wait, wait, wait, I think I found something else. And typically, the model would go again on another kind of reasoning path. And this can be one way to incentivize your model to think more. Or another term can be, OK, your time is up. Now my answer is, and it will force your model to respond. And then if you think about it, your model gives tokens in its reasoning chain, but models may think on some other space, maybe not in the language space. So there is also a line of work around having some continuous thoughts. Basically having this thinking token actually not be tokens, but be hidden representations that can be more meaningful, but also more compressed. You have a bunch of papers on this. So there's one that we linked, which is actually from 2024. But I think even a few days ago, I also saw another paper on that. So it's very much active area of research.

GRPO

And I guess just like that, I guess we saw that the way that we want to scale our model is with RL. I guess is everyone convinced of that? Did I convince you well with those arguments, or does anyone have any questions on this? And I guess here, I want you to have a takeaway, which is we want to use RL to incentivize our model to think more if we start from scratch, meaning we don't have reasoning chains. So is everyone good with this? Yeah? Kind of, yeah. So I'll take this as a yes.

So now we're going to see the algorithm that we use to perform that RL step. And you may have heard this acronym a lot. So this algorithm that we will see is called GRPO. And GRPO has been released in 2024. And it has been the go to RL algorithm for all these reasoning tasks, or for this reasoning training tasks. So what is GRPO? So GRPO stands for group relative policy optimization. And it is an RL algorithm which, very much like PPO, aims to do these two things we talked about. So the first one is to maximize advantages. And if you remember, advantage is something that tells you whether what you're producing as an answer is better than what you would expect. So this is the first part. And then the

second part is that you do not want to deviate too much from either the old model, which is the model at the previous iteration, or the base model, which is the model at SFT stage. So GRPO is also doing the same thing as PPO in that sense.

But then there is a key difference. And that difference lies in how it computes the advantage. So if you remember, PPO estimated the advantage by taking the rewards, which is at the completion level. So you know you have your prompts, and you have a response, a full response. And then based on those two elements, you have a reward. So what PPO did was take that into consideration along with something else that it was training jointly at training time, which was called the value function. And the value function objective was to try to predict what that reward was if we were to continue generating the tokens following the policy. And if you remember, the advantage here is actually, in the case of PPO, was something that we obtained based on a very complex formula that we did not see, actually. But that's called a generalized advantage estimation. That is a method to estimate those advantage. And the big limitation with that was that you had to train a value function jointly with your policy. So this big bottleneck.

And in contrast to that, GRPO says, well, we're not going to do that. We're going to take a completion. We're going to compute each reward. And we're going to compare it with the average of rewards of completions for that same prompt. So I'll give you an example. So let's suppose we have a math problem. So what we're saying here is we're going to generate multiple completions for that same prompt. And for each prompt and completion-- and the prompt is shared. So for each completion, we're going to measure, I guess, a relative measure of the reward for that completion and all the other rewards of the other completions. So this will give us an idea of how much better or how much worse is it from, I guess, the rest of the group. And so this is going to be our advantage. So the big benefit here is that we do not use value functions. We do not use that. The only thing we do is we sample not just one, but multiple completions in order to compute the average of the rewards of the group. Does this make sense?

Oh, yeah. So what would you do instead? So the question is, why would you do that. Well, the traditional RL algorithms, just for reference, they have been developed before all these LLMs became a thing. So for instance, PPO was a method that I believe is from 2017. So these quantities, they may make a lot of sense in some setups, but maybe not in the language world. And I guess here, the idea is let's try to think of something that tells you how good your completion is just in the relative sense. But here, I guess what the authors want to do is to not have to have this very expensive joint model that they have to train and think of another relative way of doing that. So I think that's the high level intuition. And I guess here, the idea is let's just sample multiple times and see how one response is comparing to the others, and then use that as some kind of baseline to put this reward into context. So for instance, you may have a math problem that's kind of easy. And if you have a high reward for that one, maybe it's because the problem was easy to start with. But if you have, let's say, very hard problem, if you have the correct answer, then you want to make sure you're incentivizing your model to upweight those tokens much more because if you find a good solution, it's a good solution to a hard problem. So you want to somehow bake that in into your advantage. So that's the intuition. So GRPO was developed a year ago. And a year ago, LLMs were everywhere. So this is also taking into consideration, I guess, what LLMs are incentivized to do in some sense. So yeah, I would think it that way. Does that help? Cool, cool. Any other question on this?

Comparison between GRPO and PPO

Well, don't worry if you have questions because we will have a walkthrough of exactly how it works. So here, we're going to go through the exact same thing, but illustrate it. So in the GRPO case, we're taking a query, and we're passing that through our policy model, which is our LLM that we're trying to train. And here, what we said was we want to generate not one, but multiple completions. Let's suppose g completions. And each of these query completions-- So you have g such pairs. You pass them through the reward model that gives you g rewards. And what you want is to compute the advantage which puts all these rewards into their context. So we saw it was basically the reward minus the average. But it's actually not exactly that. So it's reward minus the average over the standard

deviation of those rewards is how they defined it. By the way, we will see that it may not be the best way to do it, but we'll see that in a second. And we use those advantages to then tune our policy model. And while we do that, we, of course, don't want to deviate too much for from the reference model. So we have this KL divergence term as well over here. So that is GRPO.

And we're going to see right now exactly how this worked with PPO. So with PPO, if you remember, you have your prompt. You have your query. You pass that through your policy model. And you only have one completion. And you pass your prompt and your completion. So this pair, you pass it through the reward model, and this gives you a reward for that whole completion. Now, there's one thing we did not see last time, which is more of a implementation detail. But we also have a KL divergence term that is, comparing the probability of each token with the probability of the token for from the reference model. And from an implementation standpoint, we also typically incorporate that in the rewards. So we have something like a per token rewards where only the last token has the reward of the whole completion. But then everyone has also a KL divergence term. So that one is not super important here to have in mind. But it's just good to know that people implement it that way. So once you have those rewards, you also have a value function that this one is per token. This one tries to quantify the rewards if you were to continue generating the rest of the sequence following the policy. And we saw there is this method that we're not going to see. That is called generalized advantage estimation that takes in those two quantities and computes the advantage. You can think of it as some complicated formula. And you have your advantage that you use to tune the policy. And that's it, which is a complicated compared to this one.

So GRPO, you use advantages that are a result of this group computation. But then PPO, you have an advantage that is a result of rewards and the value function, which is per token. Yeah? So what are the models at stake here? So here, we have some models that we use that are, quote unquote, frozen, that are not the ones that we train. So for GRPO, we have the reference model that we do the KL divergence on. And then we have the reward model, which we trained in the past. And the same is the case for PPO. Now, one thing to note is that in the

reasoning case, we are actually not training a reward model because we know how to tell whether a solution is correct or not. We actually have a verifiable reward. So in the reasoning case, we're actually not having any reward model at all. It's the same for both. Now what is different is the models that we train. So in the GRPO case, we only train the policy model, whereas in the PPO case, we train the policy model and the value function the value model. And this is the key difference.

So until now, we've seen only drawings and illustrations. So let's see some math. So you have the GRPO loss function at the top and the PPO loss function at the bottom. They look scary. But we're going to see together some similarities and some differences. So what is one similarity? Both of them, they operate on the ratio of the probability of the current policy and the probability of the old policy. So as the first point in common. The second point in common is that both of them, they try to keep the updates within some region, not too wide. And they do that with the clipping mechanism that we saw last lecture. So I recommend that you just go through these charts that we saw together. When the advantage is positive, how the loss looks like. And when the advantage is negative. So this clipping function just allows you to not make updates that are too big, that are too wide.

But now. What is different. So in the JPO case, the KL divergence is actually something that is explicitly part of the objective function. Whereas for the PPO case, this KL divergence is typically part of the advantage. So this is more of a technical detail. But a few minutes ago I had mentioned that from an implementation standpoint, we typically incorporate the KL divergence within the rewards that are considered in the advantage computation. So that's one difference. And then the second difference is the way we compute advantages. So we saw that GRPO, we typically compute rewards for each completions, and then we compare them with the other completions from the group. Whereas for PPO, we use the rewards and the value models. So far so good?

I think I have some other things I want to talk about. But before that, I just want to open the floor if anyone has any questions. This is very technical, by the way. Very

technical, probably the hardest part of the whole class. So it's completely normal to, I guess, take some time to digest that. And if no one has any questions, I actually see this as more of an alarm for myself because this is maybe-- I don't know. So let me if there's any questions on this. And yeah, absolutely no problem to have any kind of questions. Is everything super clear? Super clear? So just tell yourself that these algorithms, they're just there to tune your model in the RL stage. PPO is heavily used in the preference tuning framework, where you try to tune your model to align it with human preferences, and is commonly used for these reasoning based training. So I think if you tell yourself that is something. And then the second thing you need to tell yourself is GRPO differs from PPO in that GRPO does not need a value function. It actually computes its advantages by comparing the rewards with respect to other completions. And then the PPO, it takes in the reward, it takes in the value, and then compares the two to get the advantage. So if this is making sense, I think this is already a very good thing. So is this making sense?

Length bias

Yeah? I would also recommend to just rewatch the recording. So we're recording this. This is very heavy and can be technical. So just rewatching it a few times I think can help. So we have 11 minutes before giving it to Shervine. And what we're going to see now is some extensions of the GRPO work that was done last year that people have done in the past few months. So this research is going back to early 2025, but it has gained enough traction for us to talk about it in this class.

So if you remember what I mentioned in the beginning of the lecture, reasoning tokens are being charged to you as a user. So you don't want to pay a lot. You don't want to pay too much. So you want your model to do what you want, but you do not want it to generate too much so that you're getting charged for it. And also from the provider side, I guess it's much better to be more efficient. So there is an incentive to see what we can optimize from the output length perspective. So if you see the training of these models, one thing that you notice is that as you train it in the RL stage, the output increases in length. So here this graph shows as a function of the RL step the average length per response. So

you see, from an empirical perspective, that the answer that the LLM is outputting is getting longer and longer. And this is mainly coming from the reasoning chain that is being more and more sophisticated. And when you compare this graph with respect to how the performance of the model improves, you see that this increase here in length is actually correlated with an increase in performance. And you might say, OK, it's good. But then there comes a stage over here where the performance-- so it's on the left-- the performance of your model tends to stabilize, but then your output length still continues increasing. And so a lot of people have looked at these charts and they're like, OK, there's something going on. So here, what we're going to do is to look at this phenomenon and see how people have tried to mitigate it.

So little Warning-- this is also a little bit technical. So please bear with me. There is one formula that I will try to explain to the best that I can. But there is some amount of math. So in order to understand what's happening, we need to look at the GRPO loss. And the loss is the one that I just mentioned here. Seems like a very complicated formula, but it's not really a complicated one. I'm going to just explain in plain terms what this means. So J of GRPO-- J means objective function. So the objective function of GRPO is to maximize this quantity, which is a function of how much your model changes with respect to the old one. So it's these ratios that you see in the slide. And you have this clipping mechanism that prevents your updates from being too wide. And then you have the KL divergence on the right, which prevents your model from being too far from the base model. The reference model. And then you have these summations here and the summations here. They correspond to you going through that process for all completions in your group. Because remember if you have a prompt you sample several times have several completions, and then you do that for every token of your output. So here, the first summation is over the indices of your group. The second summation is over the indices that are with respect to the token number in your sequence.

And so here, I guess let's just think for a second, if we were to swap-- I'm not sure if you see what I did here. I just swapped the 1 over length of output number I. Just swapped it here. I'm going to explain why I did that. So if you look at this, this is a

factor that is only a function of which output you're in. So if you're in a very long output, this will be 1 over a very long output. It will be a small number. If it's a short output, it's going to be 1 over a short number. It's going to be a big number. And the term on the right of this is token specific. So in other words, the contribution of a token with respect to the objective function depends on the output that it is in. So if the token is in a short sentence, it's going to have a higher weight bigger weight than in a longer sentence. Until now, is everyone agreeing with me? Yeah? So with that being said, I'm just going to rephrase what I just said, if you are in a short sentence, your weight will be bigger than if you're in the bigger sentence.

Now let's try to think about this in terms of the advantage. If you are in the case of a token being in an output that is of advantage, that is positive, that means that we would want to upweight the probability of this token happening again, much more for short sentences as opposed to longer ones. And at the same time, you want, when the advantage is negative-- so it's not you want. It's just that this formulation-- makes tokens that are in short outputs be down weighted even more compared to when they're in a long output. And so this is the problem. This is a bad incentive because what you're telling your model is if you have a short bad sentence, it is worse than if you have a long bad sentence. So that's the idea here. So I'm just going to just rephrase what we said until now. The fact of dividing by the length of the output is incentivizing your model to downweight even more tokens. If the tokens in a short output, it will be down weighted even more than if it's in a long output. So in other words, what you're going to do is to prefer longer bad outputs as opposed to shorter bad outputs. And this is what people are hypothesizing that is making the output length bigger and bigger.

DAPO, Dr. GRPO

And we're going to see that in a second. So what people did was do something about this. So this 1 over length of o, people wanted to do something about it. So there is one paper that came out in March, DAPO-- quite a popular paper now-- that actually equalizes this token level contributions. And so here, this normalization factor is now common to all tokens. And there is another paper that's called-- So I think it's GRPO done right. But it's Dr. GRPO So I'm not sure

how you pronounce it. But that paper actually proposes to remove the factor altogether. And when you do that, if you compare the GRPO versus the one that equalizes the token level contributions, you see that if you plot reward as a function of output length, your model will stop increasing its length again and again. So if you actually look deeper, samples that are correct are actually having a length that is matching with the GRPO one. But for incorrect solutions, here, the corrected policy is having an average length that's much lower than the ones from GRPO. So that's the bottom right charts. And that is actually making-- so this adjustment is actually making the intended consequence. So that's one very important-- I think very important-- one important modification that people typically apply.

And I'm looking at the time. In one minute, I will just tell you about some other modifications that people do. So there is a modification with respect to the standard deviation that is within the advantage formula that's biases towards in terms of the difficulty of the problem. Because if let's suppose you have a very hard problem, most of your completions will be, let's say, failures. And so your standard deviation will not be super large. So this can cause problems. And then there is another modification that people typically also consider, which is having different epsilons. And if you remember that epsilon influences how much you're able to change the policy from one iteration to another. And the idea here is that if the probability of your token is very low, it is kind of unfair to just give it a small epsilon to grow because this epsilon is multiplied. So I'm just going to write that down. And then it's going to be Shervine-- where is this? So we actually want the ratio of pi over pi all to be roughly in between those bounds roughly. So in other words, it's 1 plus epsilon times pi of old. So if this one is very low, you cannot really change your value too much. And you don't want to have an epsilon that's too high because you don't want big probabilities to suddenly go to 0. So you want to introduce an asymmetry between the lower and the upper bounds.

So this was a lot. Any questions on this? We're running short on time. We're going to have some time after the lecture in case you have any questions. And with that, I'm going to give it to Shervine.

DeepSeek R1 recipe

Thank you, Afshine, for covering all the hard parts. Now it's going to be a bit easier because we're going to see how all of this comes together. And in particular, we're going to focus at the DeepSeek papers and exactly how they used all these techniques that Afshine covered into building these reasoning models.

So what we saw at previous lectures, what, quote unquote, traditional LLMs. So you started from a pre-trained base model where you did a next token prediction on a bunch of texts of the internet. And then out of this, you proceeded to do your alignments, which was composed of SFT and then RL. So not the reasoning style RL, but the regular one. And then the SFT was instruction tuning. And we're going to see how these newer models are being trained. And I think the example of the DeepSeek R1 paper is a great one because they go into multiple stages. First, into showing how powerful can the RL stage be if you apply the verifiable rewards that Afshine mentioned. And then based on the observations of the performance that you get out of it, how to devise a full pipeline of getting a super powerful model that they call R1. So, respectively, a proof of concept and then a full reasoning model. Does that sound good? OK, great.

So now let's start with the recipe that DeepSeek folks use for the R1–Zero model. So they first started with a pre-trained model, trained on next token prediction on the latest architectures with all the bells and whistles. So you recall from previous lectures, we talked about a mixture of experts. So they have this, and they reuse a trick called multi-latent attention, MLA, that they had introduced in DeepSeek V2. And we had talked about it as well. You can see, in their architecture, they use a pre norm in their transformer blocks. So they have a typical Bayes LLM. So they perform next token prediction on it. And then this is where the fun starts. So instead of going ahead with an alignment strategy that consists of SFT, they just start with this pre-trained model just trained on next token prediction. And it hasn't seen any supervision so far. And they apply the technique that Afshine just mentioned on reasoning data. So they look at rewards. And all they incentivize the model to do is to increase the rewards

based on the outputs, like input answer that the model gives, as well as formatting. So if you have your think tokens that Afshine mentioned, then the reward on the formatting side would be given. And the paper gives exactly the templates over which they trained the model. So it's a very simple one. You have the first sentences that set up the stage that say, hey, you're discussing with the user, you're an assistant. And then the reward that I mentioned regarding formatting is explained in plain text. So it says, whatever you want to think about, just do it, but put it into think boxes. And then once you're ready to give your answer, you should wrap it under like an answer-- basically your answer blocks. And what you see here in red is what you replace by the sample prompts. And then at the end, you give the opportunity to the model to respond. Does that make sense? OK, great.

And I'm going to show you, again, a graph that Afshine had shown. So it was done on R1–Zero where they said that without any prior supervision, just having these as a reward increases scores on reasoning based benchmarks. So this one I think is on AIM. This like math one. And you see that the accuracy improves over time. And you might think, all is solved, it's great, no SFT, and we already have the best performance possible. Well, not so easy, because the authors, when they looked at the reasoning chains, they saw some issues with it of two kinds. So first, when the model was thinking about-- was emitting distinct tokens, sometimes it would mix languages. And you had also syntax issues. And you could hypothesize that this might be the case because you haven't really seen any strong supervision just before that. And you give the opportunity to the model to optimize anything it wants, but it doesn't really have, as a background, a strong supervision signal to anchor on. So this is one key challenge that we're going to see together, how they try to resolve it. Does the R1–Zero process make sense? Awesome.

So now that we have seen what R1–Zero tried to prove, the fact that with RL only, you can get reasoning level performance, now we're going to see how to adjust the challenges that we have seen into a full pipeline. So that one, they call it R1. And they start from the same basis. So they start from a V3 base, which is the pre-trained model that's DeepSeek had trained during their V3 paper. So it was

the non-reasoning version of this model. So they start from it. And then instead of starting directly to doing the RL stage that we have discussed, they try to align it in a way that will nicely try to alleviate the issues we have seen. So it starts from what they called as cold start data. They generated CoTs. during the R1–Zero stage. And these CoTs sometimes had issues. So they used humans to rewrite some of these so that all these CoTs are compliant when it comes to formatting language consistency. And then they used these pairs, prompts and then rewritten prompts, as data to train SFT on it. And I think there was no number given on the number of samples given to that stage. But from the wording of the paper, it could have been several orders of magnitude less than some of the other stages that we have seen. So potentially in the [INAUDIBLE] case of input output pairs.

So now that we have this stage ready, they continued with the RL stage that R1 0 would do. And then we're going to explore the reward function that they used. So not only we have these verifiable reward based on the response of the reasoning model output that we wanted to have, but also we have the formatting reward. And there is something new that they introduced as well to counter the effects of having poor readability in their thoughts. There was something called a language consistency reward where they use a very simple heuristic to assess whether the model indeed doesn't mix languages. So I believe it was the ratio of the tokens in the target language in the outputted chain. So they just put that in order to maximize the amount of correct language tokens. Great.

And after this first pass of RL, they continued on with SFT with a very interesting strategy. So they do it as a larger scale than the cold start one. And now they mix both reasoning and non-reasoning data in order to make the model useful to the span of cases that you as a user might solicitate the model for. And regarding non-reasoning data, they recycled some of the data that was used for their non-reasoning version of the model for V3. So about 200k pairs. And it covers the domains that we have mentioned during the instruction tuning lecture. So like a broad array of topics. And then what they did is that they added to it a lot of pairs that are directed towards reasoning. So the ratio was 3 to 1. And then the reasoning ones were generating with a method called rejection sampling. So

what they did is that they took some prompts covering these reasoning-based fields, and they used the model that had been trained so far to generate a response. And then based on the output and with some judge, they would keep some answers and reject others such that the resulting data sets would be of very high quality. So this is what they call by rejection sampling. It's just the fact of filtering out any answers that are not perfect. And the reason why I guess they did it automatically with some LLM judges and so on is given the scale of the data-- and these were heuristics that worked pretty nice on that-- you had a question? No?

So once they were done with that, they went on to a last stage that was one that might make you think about the ones that occur in non-reasoning model training pipelines. So they mix this time not only reasoning data, but also non-reasoning ones. So for the reasoning data, same kind of reward as we have seen. But on the non-reasoning one, you try to align the model on characteristics that make an LLM a helpful assistant and a harmless one. So you have this helpfulness and harmlessness components of that part of the reward. And the harmlessness was a reward that was applied on the whole tokens that were outputted. So not just the output because you want the chain of thoughts in the think section of the answer to be harmless indeed. And then the helpfulness one was more focused on what you saw at the user level. OK, great.

And just to give some idea of the results they got, so first, you notice that on reasoning-based benchmarks you have two clusters of results that appear. You have all these non reasoning based models that have some performance. And then the reasoning-based ones have unsurprisingly a performance that is much higher. And then the second thing that we observe out of these results is that R1 was pretty competitive with respect to closed source models that claim to be reasoning-based. And it was a pretty interesting moments, as Afshine had mentioned, to reproduce such a level of performance. OK, great.

So I want to finish this lecture on an interesting note of let's say you don't have a 600B MOE at your disposal. Let's say you have a smaller model, but you want to have the capabilities of these large reasoning model. How would you go about

it? So I just want to give you a reminder of how we had dealt with this topic a few lectures ago when we talked about distillation. So at that time, we were just talking about pre-training and an instruction tuning where the data we were dealing with was fixed. So for example, for next token prediction, you know the text you want the model to fit on. For SFT, you have fixed pairs. You know exactly what you want to learn. And what you did was look for each next token prediction at the output probability distribution of a teacher model. And on your student model, instead of fitting a hard label of the next token, you fit it instead the whole probability distribution of the teacher model in order to distill the knowledge of that teacher model into the student model. You all remember that? Yeah?

So now how could you go ahead with this in this case? So here, as you have seen, we don't necessarily have SFT pairs based on reasoning out of the box. So if you want to distill the knowledge of a teacher model here, the authors have gone through an interesting route that is still called distillation, but it's another flavor of it. So we use the teacher model here, R1, to generate some sample responses including thinking tokens. This is something that is done offline. And then in a second stage, you run this SFT and fit a smaller model on your distilled model, which is typically much less weights than the teacher one. So instead of fitting a probability distribution over the next token, you just fit the entire sequence. You just try to predict the same sequence of tokens that the teacher has outputted. Does this framing of distillation here make sense? Yeah? Awesome.

And I'm going to conclude here by looking at the results. So you can see that the results, once again, are very competitive with closed source alternatives that claim to be a smaller version of a reasoning model. So for example, o1–mini, you can see that the numbers here are competitive with respect to that. And then you could also wonder, but why did you go through the trouble of just distilling when you could just do the same RL techniques. Well, actually, the authors have seen that at a smaller size level, distilling the knowledge is more efficient in terms of performance than directly learning from scratch. All good? And with that, have a great weekend.